

Teaching Neural Networks Heuristics in Constraint Programming Sudoku Solvers

Sam Grouchnikov
Marietta, Georgia, USA
sam.grouchnikov@gmail.com

October 7, 2026

Abstract

Constraint satisfaction problems (CSPs) such as Sudoku are usually solved using exact, heuristic-driven algorithms that efficiently and effectively prune the search space. Recent machine learning approaches have aimed to replace these solvers entirely, often at the cost of reliability (always finding the correct answer) and interpretability. In this work, I present a hybrid framework that integrates neural-network-learned heuristics into a traditional CSP Sudoku solver. Fully-connected neural networks (FCNNs) and convolutional neural networks (CNNs) were both trained on a large dataset of boards to select the next cell to explore during recursive search based on the minimum remaining values (MRV) heuristic. The learned models were then embedded into the backtracking solver that guided variable selection during the search process. The solver was evaluated on boards with various difficulties and empty cell counts, then measured using metrics such as total recursive calls and total solve time. Experiments showed that the neural-network-guided solvers far outperformed naive baselines and showed comparable performance to that of a hard-coded, traditional algorithm. Furthermore, feature-rich versions of the neural networks far outperformed their featureless counterparts, indicating the importance of an increased context size fed into the model. While the learned heuristic solvers did not surpass the traditional algorithm, the results demonstrate that neural networks can learn heuristics and apply them to recursive CSP solvers. This work highlights not only the potential for neural networks to learn heuristics, but the overall potential of hybrid-style approaches involving both traditional recursive search and neural networks.

Introduction

Constraint Satisfaction Problems

Constraint satisfaction problems (CSPs) are a foundational subset of problems in artificial intelligence (AI) and operations research [7]. These problems are defined by a set of decision variables, constraints, and an objective function. The decision variables must be chosen such that all constraints are respected, and the objective function reaches the highest value/measure of success possible [1]. Beyond puzzles, CSPs appear in a variety of fields, including but not limited to scheduling, timetabling, and resource allocation [6, 7]. Common CSP solver approaches involve backtracking search, constraint propagation, and heuristic-based improvements to reduce the overall search space [11]. Modern approaches have recently utilized the integration of machine learning (ML) into a hybrid solving algorithm, opening the door to a plethora of research on the optimal use of ML in CSPs [16].

Sudoku

Sudoku is a popular puzzle originating in the 1900s, consisting of a 9x9 grid divided into 3x3 subgrids. The goal/objective of the puzzle is to fill every cell with a number 1-9 such that each row, column, and subgrid contains each digit exactly once. Cells are usually pre-filled to provide a starting point, and the player must logically deduce further assignments [20]. Sudoku puzzles are often rated by difficulty, and this difficulty is influenced by the number of empty cells and what values are initially filled [5]. The attraction of the puzzle comes from its contrast between simple rules and complex paths towards solutions. Despite the simplicity, Sudoku may be reduced to an NP-Complete problem, meaning general solution search algorithms may not be fully optimized to run in polynomial time. There is no “guessing” required to solve a Sudoku puzzle; each solution is reached via reasoning and logical deductions [14]. Due to this characteristic of the puzzle, it may be perfectly modeled as a CSP [9].

Sudoku as a CSP

Sudoku is naturally modeled as a CSP, which is further upheld by the assumption that all Sudoku puzzles in this study are 9x9 grids. Each cell is a decision variable, which can take on the value 1-9. Furthermore, decision variables in rows, columns, and subgrids must all take on different values. Because the constraints are highly interconnected, solving Sudoku relies on selecting robust heuristics such as minimum remaining values (MRV) and least constraining value (LCV). Constraints like these dramatically reduce the search space, leading to a more time and space-efficient solution. This study aims to teach neural networks these heuristics to compare classical algorithmic methods and modern AI-based approaches.

Related Work

Exact/Traditional Methods

An exact method is an algorithm that always finds a valid solution if one exists through an exhaustive search. Traditional Sudoku and CSP solvers fall into this category: they systematically explore a search space of possible assignments while applying logical decisions to prune infeasible areas of the search space [4, 17]. Modern exact solvers combine several complementary components - domain reductions, complete search, and deterministic inference rules - to guarantee both correctness and practical efficiency. The main benefit of the exact method is its reliability: it can be guaranteed that a solution will be found if it exists. Unfortunately,

their algorithms are worst-case exponential time (NP-Complete), meaning solve times explode drastically as puzzles get more complicated [13].

Propagation + Enumeration

A common approach in exact constraint programming solvers is summarized as propagation plus enumeration. Propagation involves the iterative reduction of variable domain sizes based on the properties of neighboring variables [10, 15]. This process shrinks the search space before any branching occurs, making it a key component of CSP solvers. Next, enumeration refers to selecting a value for a decision variable [15]. This step occurs when propagation can no longer advance, and the solver needs to make a branching choice. After each enumeration, the solver performs additional propagations. If no further propagations are possible and no solution is found, the solver backtracks to previous enumerations and explores different choices. This process guarantees either a) finding a solution or b) examining the entire search space before concluding that no solution exists. The decisions made during enumeration are crucial because they influence the search order within the space, and these choices may be guided by heuristic strategies [5, 18]. Examples of such heuristics include Least Constraining Values (LCV), which prioritize values that restrict neighbors the least, and MRV (minimum remaining values), which selects variables with the smallest remaining domain.

Sudoku as a SAT

Another common algorithmic method of solving Sudoku is the Boolean Satisfiability (SAT) formulation. In this formulation, each possible assignment to a variable is represented by a Boolean variable. The variable for a possible assignment is marked as true if and only if the cell contains the specific value. Once correctly encoded, the formula may be passed to an SAT solver to find a satisfying solution set [14]. Modern SAT solvers employ various techniques that are extremely efficient, enabling them to solve Sudoku puzzles in milliseconds. The SAT formulation is powerful due to its ability to leverage a widely researched area of algorithms and allow Sudoku to be modeled as a classic NP-Complete problem.

Dancing Links

Dancing Links (DLX) is an alternative, highly efficient method presented by Donald Knuth. This is an algorithm designed to solve the exact cover problem, which Sudoku may be reduced to. The method combines Algorithm X and doubly linked lists to remove constraints in a sparse matrix representation of the puzzle [12]. DLX excels in practice for Sudoku because the DLX algorithm is the fastest known solution to Sudoku and cover set problems, allowing correct solutions to be efficiently reached.

Machine Learning Methods

Recent scholarly research has extensively examined the implications of applying machine learning to Constraint Satisfaction Problem (CSP) solvers. Machine learning-based Sudoku CSP solvers represent a promising area of inquiry, owing to the frequent innovations emerging within this domain.

Training Neural Networks Directly

A prevalent approach within machine learning for solving Sudoku involves directly training neural networks. This generally entails representing a Sudoku board as an image (suitable for Convolutional Neural Networks) [3] or as a graph (appropriate for Graph Neural Networks) [2, 19]. Utilizing this representation, the neural

network is trained in a supervised manner, whereby it directly predicts a solution from a partially filled board. Although these methods are innovative and occasionally efficient, they do not achieve complete accuracy, thereby highlighting their current limitations and limiting practical application. Furthermore, such models often necessitate high-performance hardware for training on large datasets and exhibit notably slow inference times.

Reinforcement Learning

Another investigated methodology involves guiding CSP solvers through reinforcement learning. Through this, the solver learns which heuristics are best and is rewarded by examining a small search space. Inversely, it is punished for producing large search trees. Although not matching modern solver performance, this proof-of-concept approach demonstrated the efficacy of the application of reinforcement learning to CSP solvers [8].

Proposal

Previous approaches may largely be categorized into either a) exact algorithmic methods or b) purely machine learning methods. This study proposes a framework for hybrid solvers: constraint programming guided by neural network learned heuristics. The objective is to achieve performance comparable to traditional heuristic-based solvers without explicitly encoding the exact heuristics in the solver. By training neural nets to predict the next cell to decide, this approach aims to enable more adaptive and modern search strategies.

Modeling Sudoku as a CSP

Formally, Sudoku may be modeled as a CSP defined by the tuple

$$P = (X, D, C)$$

where:

- $X = \{x_{i,j} \mid 1 \leq i, j \leq 9\}$ is the set of decision variables, each corresponding to a cell in the Sudoku grid.
- $D = \{1, 2, \dots, 9\}$ is the domain of possible values for each variable.
- C is the set of constraints that define the relationships among variables.

Each variable $x_{i,j}$ must be assigned a value from D such that all constraints are satisfied. The set of constraints C is defined as follows:

$$\forall i \in \{1, \dots, 9\}, \quad \text{alldifferent}\{x_{i1}, x_{i2}, \dots, x_{i9}\} \quad (1)$$

$$\forall j \in \{1, \dots, 9\}, \quad \text{alldifferent}\{x_{1j}, x_{2j}, \dots, x_{9j}\} \quad (2)$$

In addition, each cell in the 3×3 subgrids must also contain distinct values, which introduces the following constraint:

$$\forall i, j, \quad \text{AllDifferent} \left\{ \begin{array}{l} x_{ij}, x_{i(j+1)}, x_{i(j+2)}, \\ x_{(i+1)j}, x_{(i+1)(j+1)}, x_{(i+1)(j+2)}, \\ x_{(i+2)j}, x_{(i+2)(j+1)}, x_{(i+2)(j+2)} \end{array} \right\} \quad (3)$$

This formulation was extracted from [10]

Methods

The methods conducted may be split into two major parts: (1) training the neural network and (2) integrating the neural net into a CSP solver specifically for Sudoku.

Model Training

Two neural network types were utilized in this study: traditional fully-connected networks (FCNNs) and convolutional neural networks (CNNs).

Training Data

The training dataset contained a set of 9 million 9 by 9 unsolved Sudoku puzzles. These unsolved puzzles contain varying amounts of empty cells, ranging from 4 to 58. This is highlighted in Figure 1, which displays the number of boards for each empty cell count. A y-limit cut chart is also presented to more clearly indicate empty cell boards with less frequent appearances.

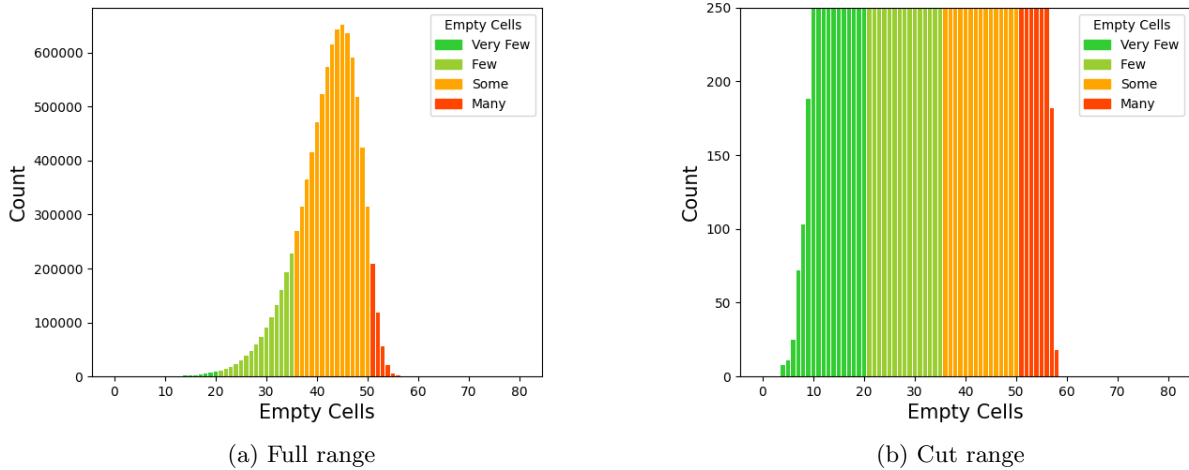


Figure 1: Counts of each empty cell count in the dataset.

This dataset was then given a label from 1 to 81 indicating which cell index gives the most promising path for a quick search space exploration. This labeling was done based on a minimum remaining values (MRV) heuristic, which chooses the next cell with the smallest domain (minimum remaining values possible). Cells with the same MRV were then differentiated with a tie-break, which was determined by the amount

of cells empty in the row, column, and 3x3 grid. The cell with the smallest sum of remaining values in the tie-break metric was chosen. If multiple cells had the same lowest tiebreak value, the first one was chosen.

Feature Expansion

To aid the models with learning relevant features from the data, the original 9x9 board was extended into a feature-rich 9x9x30 representation. In this representation, each of the 81 cells had a vector of 30 features attached. These features included a one-hot encoding of each cell’s state, data on empty cells remaining in neighboring areas, and total domain size of the current state. This method was tested against simply feeding the 9x9 grid through the neural network.

Neural Network Training

A fully-connected neural network (FCNN) and a convolutional neural network (CNN) were implemented using PyTorch Lightning, with training metrics logged with Weights & Biases. The neural networks of respective types contained the same number of layers to ensure experimental validity. These architectural aspects as highlighted in Figure 2.

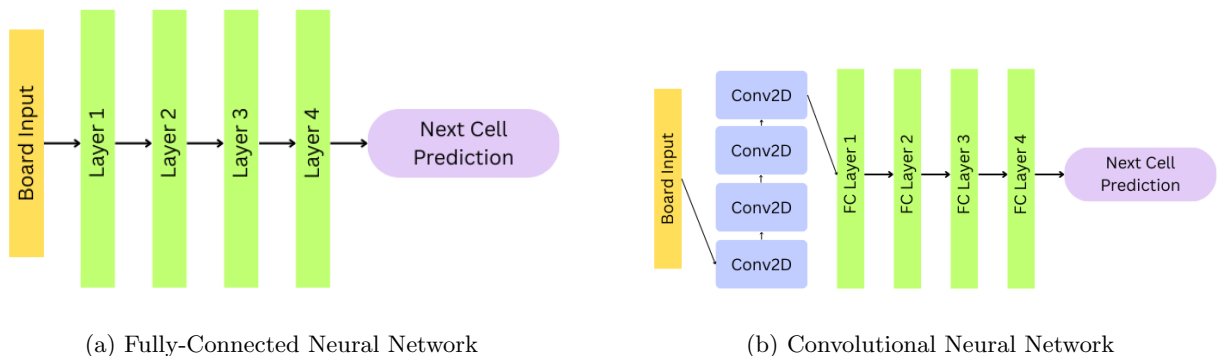


Figure 2: Comparison of the two neural network architectures.

To compare the effect of model size on final accuracy, layer sizes were experimentally changed. Ideally, smaller models should have comparable performance with the larger ones for best forward pass efficiency.

Fully-Connected Neural Network

The hidden layer sizes were all identical throughout the neural network. This means when the hidden layer size was changed, each hidden layer had the same size update. The sizes along with their overall parameter counts are shown in Table 1.

Convolutional Neural Network

The fully connected layer sizes were kept constant throughout all tests, with a size of 256. This size gives a favorable balance of model size and forward pass efficiency. The changed model aspect was the number of output channels from the final convolutional layer, along with the input size of the first hidden layer to ensure legal matrix multiplications. The CNN model sizes are shown in Table 2.

Table 1: FCNN Hidden Layer Sizes

Hidden Size	Param #	Expanded Param #
128	53k	354k
256	140k	741k
512	607k	1.8m
1028	2.3m	4.7m

Table 2: CNN Output Layer Sizes

Conv Out Size	Param #	Expanded Param #
1	247k	263k
2	268k	284k
4	309k	326k
8	393k	409k

Model Training Setup

All important model aspects are highlighted below in Table 3.

Table 3: Model Training Setup

Category	Parameter	Value
Model Architecture Training Configuration	Activation	ReLU
	Optimizer	Adam
	Learning Rate	0.001
	Dropout Rate	0.2
	Loss Function	Cross-Entropy
	Epochs/Batch Size	10/128
Dataset Splitting	Train/Test/Validation Ratio	70/10/20

The final model output was a 9x9 tensor with probabilities of each next cell being chosen. The chosen cell was the one with the highest probability, obtained using *argmax*. The final evaluation metric was prediction accuracy, in which the model was considered to predict correctly if it made an exact match with the ground truth labels.

Solver Integration

The solver operates using a traditional backtracking approach, pruning the search space as cells are assigned. Once a feasible solution is reached, the algorithm terminates. The cornerstone of the algorithm is the next cell selection, obtained from the trained neural network. This is highlighted in Algorithm 1.

When each assignment is made, domains of neighboring cells will naturally be changed, specifically made smaller. This assists with search tree pruning. Multiple domain propagation sequences are required, as

some propagations will lead to further ones not seen in the original search. This process is highlighted in Algorithm 2.

A recursive search is initiated on run, where the feasible search space is explored. In contrast to naive search, this approach selects the model-chosen cell to start exploring, ideally only searching "probable" areas of the search space. This process is highlighted in Algorithm 3

Evaluation

To evaluate the selections of the neural networks, a traditional heuristic-based algorithm was compared to the proposed one. The key difference is the *SelectNextCell* function, in which the traditional was computed based on variable domains and filled cell positions. This computation yields the same results as the algorithm used to generate ground-truth scores in the dataset. Two metrics were used for comparisons: time and total recursive calls. Time is defined as the total number of second taken to solve the entire board (model loading time was not considered for this calculation). Total recursive calls is defined as the total number of times the search function called itself during the search space exploration. An algorithm that effectively prunes the search space in turn significantly reduces total amount of recursive calls, and thus the total run time of the algorithm. Furthermore, a set of boards with empty cell counts ranging from 59-62 were added to the test set to determine if the neural nets could generalize to unseen empty cell counts.

Results

Model Training

All runs were conducted on a cluster of 3 Nvidia RTX 3090 GPUs, resulting in a total training time of around one hour for each model variant. To ensure correct model weight updates, loss and accuracy values were plotted as a function of total global step. Due to frequent overlaps between plotted values, an exponential moving average (EMA) of each line was boldly plotted over each original metric line. This is highlighted in Figures 3 and 4. The mapping of the legend names to their full description is shown in Table 4

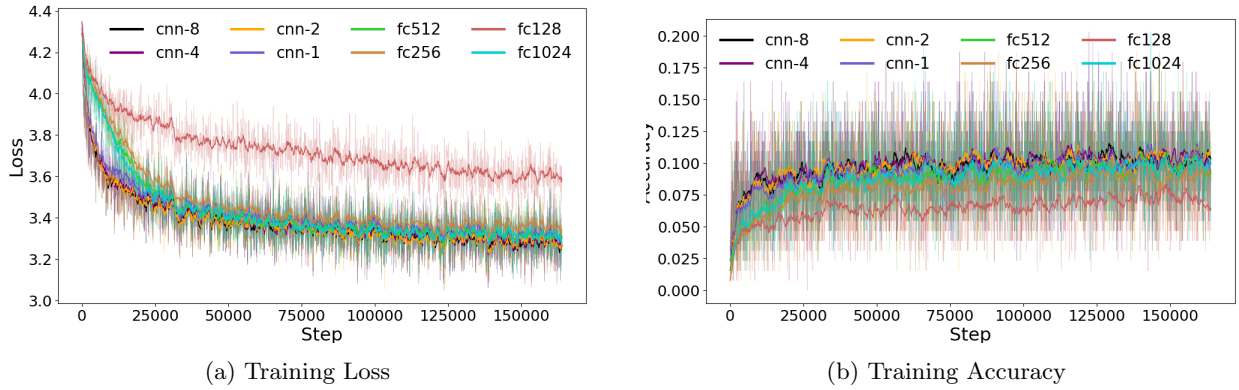


Figure 3: Train loss and accuracy for the featureless neural networks.

As expected, the training loss and accuracy both converged exponentially, indicating the model was correctly trained. However, the training accuracy did show high variation, suggesting the featureless neural

network struggled with capturing board aspects beyond simply filled cells. This variation was only smoothed through a heavy weighted EMA.

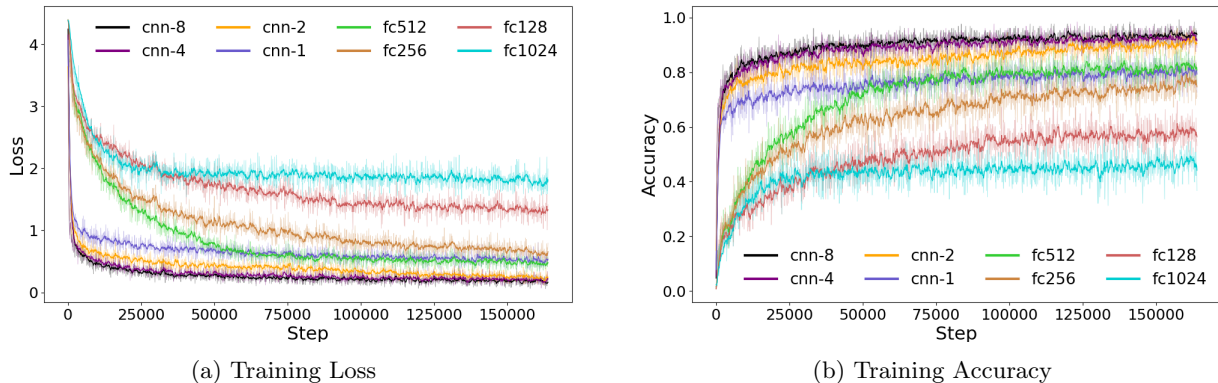


Figure 4: Train loss and accuracy for the feature-rich neural networks.

Table 4: Mapping of legend names to model descriptions

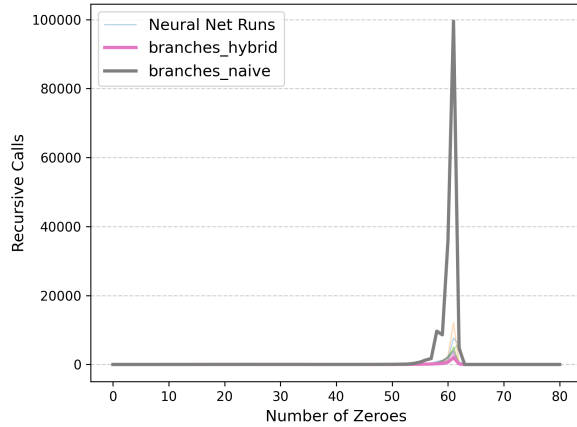
FCNN Variation	Description	CNN Variation	Description
fc128	FCNN w/ hidden size 128	cnn-1	CNN w/ 1 out channel
fc256	FCNN w/ hidden size 256	cnn-2	CNN w/ 2 out channels
fc512	FCNN w/ hidden size 512	cnn-4	CNN w/ 4 out channels
fc1024	FCNN w/ hidden size 1024	cnn-8	CNN w/ 8 out channels

As expected, giving the neural networks more features of the board beyond filled cells helped significantly. Some model variants such as cnn-8 and cnn-4 reached the near perfect prediction accuracy. This indicates neural networks can learn useful heuristics in a general constraint programming solver.

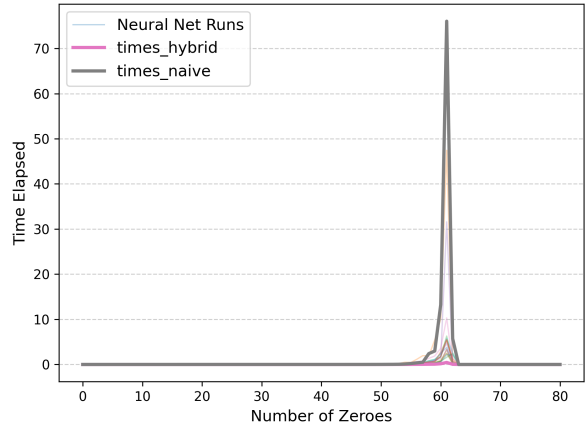
Algorithmic Evaluations

To evaluate the benefits of using neural-network guided solvers, all model-guided solvers were first compared to both a hard-coded heuristic based approach (same heuristic learned by the neural nets) and a naive approach. The naive approach selected the next cell based of pure randomness, choosing from any of the currently non-filled cells. The recursive calls and total times were then plotted by index (empty cell count) to determine if neural net guided solvers outperformed naive approaches. This is highlighted below in Figure 5.

As expected, the naive baseline algorithm conducted many more recursive calls and took more time to solve than any of the other methods. Additionally as expected, the hard-coded heuristic based algorithm outperformed all others in terms of calls and solving time. This behavior is expected as no neural network was able to reach 100% test accuracy. Then to examine exactly which factors affected final algorithm performance, the data was analyzed in three different perspectives. This included feature-rich vs featureless based models, CNNs vs FCNNs, and size of the model used.



(a) Average recursive calls per empty cell count



(b) Average solving time per empty cell count

Figure 5: Comparison of recursive calls and solving times across various algorithmic approaches

Feature-rich vs Feature-less Models

Recursive calls and time elapsed data items per empty cell count were grouped into 3 sets: 0-40, 41-55, 56-62. This was done due to the relative similarity in values in these groups, and the overall volume of data accumulated across tests. FL denotes featureless neural networks, and FR denotes feature-rich neural networks. The comparison results are shown in Tables 5, 6.

Table 5: Comparisons of feature-rich and featureless neural network recursive calls

Empty cells	FL CNN	FR CNN	FL FCNN	FR FCNN
0-40	12.864	10.223	12.687	10.459
41-55	29.113	19.57	36.981	17.207
56-62	1109.843	1013.168	1723.434	567.873

Table 6: Comparisons of feature-rich and featureless neural network time (s) elapsed

Empty cells	FL CNN	FR CNN	FL FCNN	FR FCNN
0-40	0.0344	0.0357	0.0101	0.0436
41-55	0.06596	0.0507	0.0235	0.0592
56-62	1.701	2.933	71.809	1.580

As shown the the tables and in accordance with model training results, feature-rich neural networks far outperform their featureless counterparts. Lower empty cell counts saw near 20% reductions in recursive calls and time, while the highest empty cell counts saw elapsed times reduced by a factor of three. This implicates the importance of the addition of feature maps into the neural network input, and that the additional forward

pass time from this is far out-shadowed by the total reduction in solving. This is further demonstrated in Figure 6 where total recursive calls are plotted as a function of time for both CNNs and FCNNs.

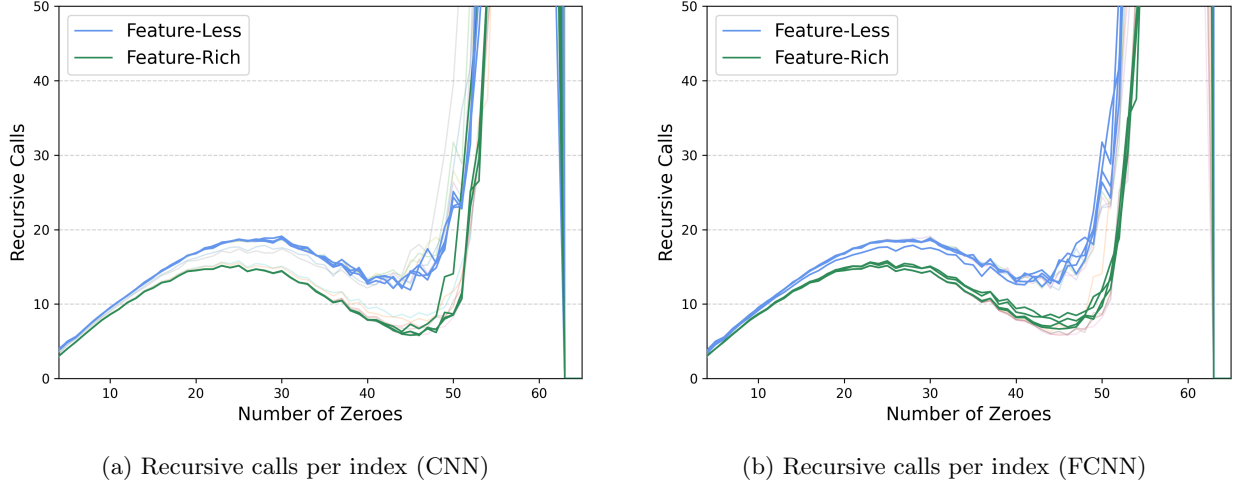


Figure 6: Comparison of recursive calls and solving times across various algorithmic approaches

Model Sizes

To determine the importance of model size on algorithm performance, data was extracted for average recursive calls and solving times relative to the model sizes. This was done for both feature-rich and featureless based solvers to ensure the only determining factor was model sizes. The comparisons are shown below in Figure 7.

It can be deduced from the table that in general, model size itself introduces no clear trends into overall algorithmic performance, especially in boards with fewer empty cells. The main exception is presented in boards with at least 56 empty cells, where larger models do somewhat help the solver conduct fewer total recursive calls. Despite this, FC-1024-based solver has worse performance than its 512-sized counterpart, showing this either be an outlier or showing the decreasing behavior in other FC variants occurred due to random chance.

FC Neural Network vs CNN

Finally, to elucidate the importance of the type of neural network used, comparisons were made between the two aforementioned types. This is shown below in tables 8a and 8b.

Similarly to the last comparison, no clear trends emerged across tests, indicating that the actual model architecture seemed to have no clear effect on performance. This was unexpected, as CNNs performed much better in terms of pure accuracy in the training portion. This is likely due to the few outliers where search spaces exploded due to random chance, which is expected given the randomness of recursive search's nature. Additionally, the CNN-1 model seemed to be an outlier where recursive calls and time elapsed exploded due to unknown reasons, further skewing the data.

(a) Recursive calls per index					(b) Solver time per index (s)				
Index	Model	Size	FL	FR	Index	Model	Size	FL	FR
0-40	cnn	1	12.832	10.296	0-40	cnn	1	0.0197	0.0814
0-40	cnn	2	12.902	10.276	0-40	cnn	2	0.018	0.021
0-40	cnn	4	12.894	10.273	0-40	cnn	4	0.029	0.021
0-40	cnn	8	12.815	10.278	0-40	cnn	8	0.070	0.021
0-40	fc	128	12.230	10.630	0-40	fc	128	0.005	0.030
0-40	fc	256	12.830	10.327	0-40	fc	256	0.008	0.049
0-40	fc	512	12.834	10.319	0-40	fc	512	0.016	0.049
0-40	fc	1024	12.818	10.732	0-40	fc	1024	0.012	0.047
40-55	cnn	1	22.620	22.123	40-55	cnn	1	0.040	0.108
40-55	cnn	2	22.696	13.033	40-55	cnn	2	0.027	0.020
40-55	cnn	4	22.172	13.355	40-55	cnn	4	0.044	0.020
40-55	cnn	8	22.770	12.860	40-55	cnn	8	0.108	0.020
40-55	fc	128	28.478	13.678	40-55	fc	128	0.011	0.046
40-55	fc	256	30.734	13.252	40-55	fc	256	0.018	0.049
40-55	fc	512	24.590	12.870	40-55	fc	512	0.026	0.047
40-55	fc	1024	27.000	14.537	40-55	fc	1024	0.022	0.057
56-62	cnn	1	1071.323	2117.262	56-62	cnn	1	1.087	8.590
56-62	cnn	2	1059.967	501.273	56-62	cnn	2	1.338	0.615
56-62	cnn	4	989.104	500.467	56-62	cnn	4	1.244	0.527
56-62	cnn	8	819.653	462.733	56-62	cnn	8	2.388	0.608
56-62	fc	128	2260.713	554.338	56-62	fc	128	0.703	1.385
56-62	fc	256	1841.688	478.725	56-62	fc	256	0.709	1.321
56-62	fc	512	893.891	433.743	56-62	fc	512	4.347	1.199
56-62	fc	1024	1111.836	552.268	56-62	fc	1024	0.610	1.710

Figure 7: Comparisons of feature-rich (FR) and featureless (FL) neural network performance

Conclusions

Teaching Neural Network Heuristics

Overall, with certain architectural decisions, neural networks can learn heuristics for next cell decision-making in CSP solvers. The most prevalent conclusion that may be drawn from the results is the importance of adding additional input features to aid with the model’s contextual understanding. This can be deduced from the fact that even the worst performing feature-rich model outperformed all featureless models by a factor of over three. Furthermore, results show it is highly beneficial to utilize the CNN architecture instead of a plain FCNN, as a CNN has the potential to learn spacial relationships more rubustly.

(a) Average recursive calls				(b) Average time elapsed (s)			
Empty Cells	Model	FL	FR	Empty Cells	Model	FL	FR
0-40	CNN	12.861	10.281	0-40	CNN	0.034	0.036
0-40	FCNN	12.679	10.503	0-40	FCNN	0.010	0.044
40-55	CNN	22.565	15.343	40-55	CNN	0.055	0.042
40-55	FCNN	27.701	13.584	40-55	FCNN	0.019	0.050
56-62	CNN	985.012	895.435	56-62	CNN	1.514	2.587
56-62	FCNN	1527.033	504.769	56-62	FCNN	1.592	1.404

Figure 8: Comparison of featureless (FL) and feature-rich (FR) CNNs vs FCNNs across empty-cell count groups

Model Sizes

For pure prediction accuracy, model size did yield a clear trend for CNNs. The CNNs with more output channels (and therefore more parameters) did achieve higher accuracy, whereas the CNN with only 1 output channel yielded the lowest accuracy. This implicates the importance of model size for CNNs, and is in accordance with expectations given the general nature of neural networks. On the other hand, no clear trends emerged for the FCNN. This was unexpected, especially given that the largest FCNN ended up performing the worst of all models. This may be due to a fatal training error during the training loop that pushed gradients too far in the wrong direction, but may also be from the stochastic nature of optimization algorithms.

Solver Integrations

Overall, neural-network-based solver variants far outperformed a naive implementation, proving their practical accuracy and benefits over such baselines. These benefits were especially noticeable in the total recursive calls comparison, as the nn-based solvers explored many fewer possibilities. This reduction in search space had a factor in the tens for each type of nn-based solver, even the worst performing one. In addition to this, total solver times were much lower for nn-based solvers after a certain number of empty cells was reached. This proves that even with the computationally heavy forward passes for each prediction, nn-based solvers made up for this with their search space reduction efficiency.

Important Model Architectural Decisions

Across 3 different types of experiments (model size, model type, model feature inputs), clear trends emerged in the importance of architectural aspects of the neural network the solver was based on. The model type seemed to have no clear effect on final performance, an unexpected behavior given the clear superiority in prediction accuracy presented by the CNN variants. This is likely due to the randomness presented in recursive search, and the lack of trend may be eliminated across larger test dataset sizes. Furthermore, model size seemed to also have little effect on final performance for lower empty cell counts. This is expected as the search space is already small, so slightly better next cell predictions are unlikely to have a large effect on overall performance. The opposite happened for larger empty cell counts however, as larger models outperformed their smaller counterparts. This is expected as with more empty cells, the search space is

larger and thus it is even more important to prune this search space when possible. Finally, feature inputs had the largest effect on solver performance. This is especially true for total recursive calls, as feature-rich-model-based solvers consistently outperformed the featureless ones.

Success Metric Discussion

Due to the heavy calculations required for each next cell prediction, no clear trends emerged for time elapsed for any solvers. This was expected as especially on a CPU, neural networks are inefficient to conduct forward passes for, overshadowing the effect of better-pruned search spaces. Total recursive calls is a much better metric of success, as it is more heavily based on the exploration done by the solver, and is independent of available hardware.

Limitations

One major limitation presented is the inability for a nn-based solver to ever surpass the performance of the optimized, hard-coded solver. This is due to the fact that the neural networks were trained to make these hard-coded predictions, and thus by inspection can never outperform the hard-coded solver. Another major limitation was the size of the test dataset used. Running many solver test experiments on a CPU was computationally expensive, and thus the total test dataset size was relatively small. Finally, an extremely important limitation was the high likelihood of unintended, unavoidable data leakage. As a solver progresses, it is likely that its partially solved state mimics that of a state seen during model training. Although this means predictions were likely not always done on unseen data, this is an unavoidable problem without careful data formula, a task that lies outside the scope of this study.

Future Works

Due to time and resource constraints, graph neural networks (GNNs) were not utilized. GNNs are a standard for solving interconnected puzzles such as a Sudoku, and their use is a crucial area of future investigation. It is important to continue scaling down the size of a model to be comparable in terms of time elapsed to hard-coded solvers, a path that is likely most feasible with GNNs. Another important area of future investigation is the analysis of the aforementioned unavoidable data leakage. Although it cannot be fully eliminated, the extent to which the leakage affected final results may be analyzed to determine overall validity and reliability of the method as a whole. Finally and most importantly, the decisions made by the neural networks must be interpreted. The solver made many neural-network-based decisions while searching for solutions, and these decisions must be interpreted to determine how and why the neural network selects certain cells. Interpretability is a crucial area of future investigation, and our understanding of the solver is incomplete without such an analysis.

References

- [1] Constraint satisfaction problems (CSP) in artificial intelligence. <https://www.geeksforgeeks.org/artificial-intelligence/constraint-satisfaction-problems-csp-in-artificial-intelligence/>, June 2023. Accessed: 2025-11-4.
- [2] Extending GraphSAGE to solve sudoku. <https://dev.to/sammoorsmith/extending-graphsage-to-solve-sudoku-2d8j>, March 2023. Accessed: 2025-11-4.
- [3] Sudoku solver using TensorFlow. <https://www.geeksforgeeks.org/deep-learning/sudoku-solver-using-tensorflow/>, December 2023. Accessed: 2025-11-4.
- [4] Marlene Arangú and Miguel Á Salido. A fine-grained arc-consistency algorithm for non-normalized constraint satisfaction problems. *International Journal of Applied Mathematics and Computer Science*, November 2011.
- [5] Apekshya Bhattarai, Dinisha Uprety, Pooja Pathak, Safal Narshing Shrestha, Salina Narkarmi, and Sanjog Sigdel. A study of sudoku solving algorithms: Backtracking and heuristic. *arXiv [cs.LO]*, July 2025.
- [6] Mahdi Bidar and Malek Mouhoub. Nature-inspired algorithms for solving weighted constraint satisfaction problems. December 2022.
- [7] Sally C Brailsford, Chris N Potts, and Barbara M Smith. Constraint satisfaction problems: Algorithms and applications. *Eur. J. Oper. Res.*, 119(3):557–581, December 1999.
- [8] Félix Chalumeau, Ilan Coulon, Quentin Cappart, and Louis-Martin Rousseau. SeaPearl: A constraint programming solver guided by reinforcement learning. *arXiv [cs.LG]*, February 2021.
- [9] Aaron Christensen and Gabe Emerson. Sudoku puzzles as a constraint satisfaction problem. <https://www.semanticscholar.org/paper/2c43c967e84cb1ecda4908d6021babc8d4bbe68d>, 2007. Accessed: 2025-12-15.
- [10] Broderick Crawford, Mary Aranda, Carlos Castro, and Eric Monfroy. Using constraint programming to solve sudoku puzzles. In *2008 Third International Conference on Convergence and Hybrid Information Technology*, volume 2, pages 926–931. IEEE, November 2008.
- [11] Spencer K L Fung, D Zheng, Ho-Fung Leung, Jimmy Ho-Man Lee, and H Chun. A framework for guided complete search for solving constraint satisfaction problems and some of its instances. *16th IEEE International Conference on Tools with Artificial Intelligence*, November 2004.
- [12] Donald E Knuth. Dancing links. *arXiv [cs.DS]*, November 2000.
- [13] Michael Lampis. Review of exact exponential algorithms by fedor V. fomin and dieter kratsch. *ACM SIGACT News*, 44(2):17–21, June 2013.
- [14] Inês Lynce and Joël Ouaknine. Sudoku as a SAT problem. <https://sat.inesc-id.pt/~ines/publications/aimath06.pdf>. Accessed: 2025-11-6.

- [15] É Monfroy, C Olarte, and C Rueda. Exploring process calculi as a mechanism to define dynamic enumeration strategies in constraint programming. December 2005.
- [16] A Popescu, S Polat-Erdeniz, A Felfernig, and M Uta. An overview of machine learning techniques in constraint solving. December 2021.
- [17] Helmut Simonis. Sudoku as a constraint problem. https://ai.dmi.unibas.ch/_files/teaching/fs21/ai/material/ai26-simonis-cp2005ws.pdf. Accessed: 2025-11-4.
- [18] Ricardo Soto, Broderick Crawford, Rodrigo Olivares, Rodrigo Herrera, Franklin Johnson, and Fernando Paredes. Enumeration strategies to solve constraint satisfaction problems: Performance evaluation. In *2015 10th Iberian Conference on Information Systems and Technologies (CISTI)*, pages 1–6. IEEE, June 2015.
- [19] Andy Tran, Joël Mathys, Florian Grötschla, and Roger Wattenhofer. Enhancing GNNs: An exploration of iterative solving and augmentation techniques.
- [20] Hao Wang, Yu-Wen Wang, and Chuen-Tsai Sun. Rating logic puzzle difficulty automatically in a human perspective. In *DiGRA Digital Library*. Digital Games Research Association DiGRA, January 2012.

Appendix

Algorithm 1 SelectNextCell(board, domains)

```
1:  $x \leftarrow \text{Preprocess}(\text{board}, \text{domains})$ 
2:  $p \leftarrow \text{model}(x)$ 
3:  $idx \leftarrow \arg \max(p)$ 
4:  $idx2d \leftarrow (idx \div 9, idx \bmod 9)$ 
5: return  $idx2d$ 
```

Algorithm 2 AssignBoard(board, domains, r, c, value)

```
1:  $board\_copy \leftarrow \text{copy of } board$ 
2:  $domains\_copy \leftarrow \text{copy of } domains$ 
3:  $board\_copy[r, c] \leftarrow \text{value}$ 
4: Restrict  $domains\_copy[r, c]$  to value
5:  $queue \leftarrow \{(r, c)\}$ 
6: while  $queue \neq \emptyset$  do
7:    $(cr, cc) \leftarrow queue.pop()$ 
8:    $assigned\_val \leftarrow [1..9]$  value assigned at  $(cr, cc)$ 
9:   for all  $cell$  in row  $cr$ , column  $cc$ , and  $3 \times 3$  box do
10:    if  $cell \neq (cr, cc)$  and  $board\_copy[cell] = 0$  and  $assigned\_val \in domains\_copy[cell]$  then
11:      Remove  $assigned\_val$  from domain
12:      if one value left in  $domains\_copy[cell]$  then
13:        Assign that value to  $board\_copy[cell]$ 
14:         $queue \leftarrow queue \cup \{cell\}$ 
15:      end if
16:    end if
17:  end for
18: end while
19: return  $board\_copy, domains\_copy$ 
```

Algorithm 3 Solve(board, domains)

```
1: if board is complete then
2:   return board
3: end if
4:  $(r, c) \leftarrow \text{SelectNextCell}(\text{board}, \text{domains})$ 
5: for all value  $\in \text{domains}[r, c]$  do
6:   if value gives legal assignment then
7:     BoardAssign(board, domains,  $r, c$ , value)
8:     result  $\leftarrow \text{Solve}(\text{board}, \text{domains})$ 
9:     if result is not failure then
10:      return result
11:   end if
12:   UndoAssign(board, domains,  $r, c$ )
13: end if
14: end for
15: return failure
```
